# File IO Handling in Python

Python supports file handling and allows users to handle files i.e., to read and write files, along with many other file handling options, to operate on files. Python treats file differently as text or binary and this is important. Each line of code includes a sequence of characters and they form text file. Each line of a file is terminated with a special character, called the EOL or End of Line characters like comma {,} or newline character. It ends the current line and tells the interpreter a new one has begun.

# File Handling

The key function for working with files in Python is the `open()` function.
We use **open ()** function in Python to open a file in read or write mode
The `open()` function takes two parameters; *filename*, and *mode*.
There are four different methods (modes) for opening a file:
`"r"` - Read - Default value. Opens a file for reading, error if the file does not exist
`"a"` - Append - Opens a file for appending, creates the file if it does not exist
`"w"` - Write - Opens a file for writing, creates the file if it does not exist
`"x"` - Create - Creates the specified file, returns an error if the file exists

In addition you can specify if the file should be handled as binary or text mode
`"t"` - Text - Default value. Text mode
`"b"` - Binary - Binary mode (e.g. images)

# Binary vs Text Files in Python

There are two separate types of files that Python handles: *binary* and *text* files.
Most files that you use during your normal computer use are actually **binary files**, not text.
Microsoft Word .doc file is actually a binary file, even if it just has text in it. Other examples of binary files include:

- Image files including `.jpg`, `.png`, `.bmp`, `.gif`, etc.
- Database files including `.mdb`, `.frm`, and `.sqlite`
- Documents including `.doc`, `.xls`, `.pdf`, and others.

That's because these files all have requirements for special handling and require a specific type of software to op
For example, you need Excel to open an `.xls` file, and a database program to open a `.sqlite` file.
A **text file** on the other hand, has no specific encoding and can be opened by a standard text editor
without any special handling.
Still, every text file must adhere to a set of rules:

- Text files have to be readable as is. They can (and often do) contain a lot of special encoding,
- especially in HTML or other markup languages, but you'll still be able to tell what it says
- Data in a text file is organized by lines. In most cases, each line is a distinct element,
- whether it's a line of instruction or a command.

# Syntax

- To open a file for reading it is enough to specify the name of the file:
- f = open("demofile.txt")
- The code above is the same as:
- f = open("demofile.txt", "rt")

- The `open()` function returns a file object, which has a `read()` method for reading the content of the file:
- f = open("demofile.txt", "r")
  print(f.read())

# Read Only Parts of the File

By default the `read()` method returns the whole text, but you can also specify how many characters you want to return:

```
open "demofile.txt"  "r"
print(f.read(5))
```

- Read Lines

You can return one line by using the `readline()` method:

```
f = open("demofile.txt", "r")
for x in f:
  print(x)
```

Close Files:

It is a good practice to always close the file when you are done with it.

```
f = open("demofile.txt", "r")
print(f.readline())
f.close()
```

You should always close your files, in some cases,
due to buffering, changes made to a file may not show
until you close the file.

**Python File Write:**

# Write to an Existing File

To write to an existing file, you must add a parameter to the `open()` function
`"a"` - Append - will append to the end of the file
`"w"` - Write - will overwrite any existing content

```
# Python code to create a file
file = open('geek.txt','w')
file.write("This is the write command")
file.write("It allows us to write in a particular
file")
file.close()
```

## Create a New File

To create a new file in Python, use the `open()` method, with one o
`"x"` - Create - will create a file, returns an error if the file exist
`"a"` - Append - will create a file if the specified file does not exist
`"w"` - Write - will create a file if the specified file does not exist

Create a file called myfile.txt

```
f = open("myfile.txt", "x")
```

a new empty file is created!

# Python Delete File

## Delete a File

To delete a file, you must import the OS module, and run its `os.remove()` function:

```
import os
os.remove("demofile.txt")
```

```
import os
if os.path.exists("demofile.txt"):
  os.remove("demofile.txt")
else:
  print("The file does not exist")
```

**To Delete a Complete Folder**

```
import os
os.rmdir("myfolder")
```

```
# Python code to illustrate split() function
with open("file.text", "r") as file:
    data = file.readlines()
    for line in data:
        word = line.split()
        print word
```

| Access Modes | Description |
| --- | --- |
| r | Opens a file for reading only. |
| rb | Opens a file for reading only in binary format. |
| r+ | Opens a file for both reading and writing. |
| rb+ | Opens a file for both reading and writing in binary format. |
| w | Opens a file for writing only. |
| wb | Opens a file for writing only in binary format. |
| w+ | Opens a file for both writing and reading. |
| wb+ | Opens a file for both writing and reading in |